CAPITAL FLOWS RESEARCH

STIR REPLICATION PLAYBOOK

BUILD A CME-FEDWATCH-STYLE FED FUNDS DASHBOARD

FOR SUBSCRIBERS

WHAT YOU'LL BUILD

A schematic preview of the two-tab US STIR dashboard you'll re-create from this playbook.

WHAT YOU'LL BUILD

A two-tab short-term-interest-rate dashboard for the United States built entirely from publicly listed CME futures and publicly published reference rates.

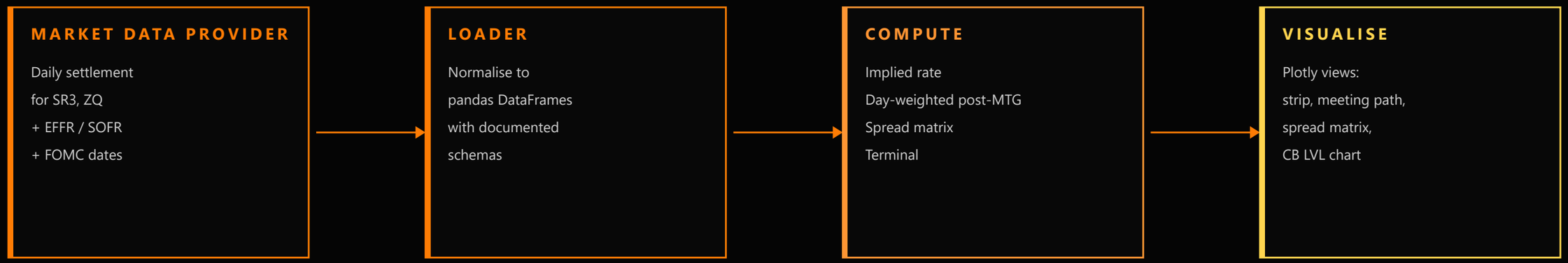
PRODUCTS tab — view the SOFR or Fed Funds futures strip with a single toggle, against today's effective federal funds rate.

MEETINGS tab — three sub-views derived from Fed Funds futures: a **strip** with implied post-meeting rates, a **spreads** calendar matrix, and a **CB LVL** chart that snaps reference rails to 25 bp policy increments.

The methodology, formulas and a runnable Plotly-based reference implementation live entirely in this single PDF — full code in the appendix at the back. Paste the PDF into Claude Code, point it at your data source, and the rest is mechanical.

ARCHITECTURE

A four-stage pipeline. Each box is independent — you can swap any stage without touching the others.



PIPELINE NOTES

- **Provider-agnostic.** Anything that returns daily settlement prices for CME SR3 and ZQ contracts will work — Refinitiv, Polygon, IBKR, CME DataMine, your own broker feed.
- **Two reference rates.** The effective federal funds rate (EFFR) and the secured overnight financing rate (SOFR) are both published by the New York Fed at `markets.newyorkfed.org`.
- **Calendar.** The FOMC meeting schedule is published at `federalreserve.gov`; you only need a list of meeting dates.
- **No build dependencies beyond pandas and Plotly.** Everything in this playbook fits in a single notebook.

DATA INPUTS

Five inputs total. CME ticker roots are the public industry-standard identifiers — translate them to your provider's symbology.

REQUIRED MARKET DATA

INSTRUMENT	CME ROOT	WHAT IT IS	USED BY
SR3	SR3M · SR3U · SR3Z · SR3H	Three-month SOFR future. Quarterly listings (March, June, September, December).	Products tab — SOFR strip
ZQ	ZQF · ZQG · ZQH · ZQJ…	Thirty-day federal funds future. Monthly listings (every calendar month).	Products tab — FF strip; Meetings tab — strip / spreads / CB LVL
EFFR	NY Fed publication	Effective federal funds rate, published each business day for the prior session.	OCR baseline reference (dashed line on every chart)
SOFR	NY Fed publication	Secured overnight financing rate, published each business day for the prior session.	Spot overlay on the SOFR strip view (basis vs EFFR)
FOMC	Federal Reserve schedule	Calendar of FOMC meeting end-dates for the next ~18 months.	Day-weighting algorithm in the Meetings tab

PROVIDER NOTES

- Settlement prices are sufficient — you don't need intraday tick data. Daily close is what every panel here is built on.
- For each contract you need its **expiry month** and **settlement price**. Open interest and volume are nice-to-haves but not required.
- The two reference rates (EFFR, SOFR) come straight from the New York Fed's public CSV downloads — no subscription required.
- The FOMC schedule is a hand-maintained list of ~8 dates per year. Hard-code it; refresh annually.

EXPECTED SCHEMAS

Three lightweight DataFrames. Hand these to every downstream function.

PYTHON

```python
from dataclasses import dataclass
from datetime import date
import pandas as pd


@dataclass
class Contract:
    symbol:    str       # e.g. "SR3M5", "ZQK6"
    root:      str       # "SR3" or "ZQ"
    expiry:    date      # contract expiration
    settle:    float     # last settlement price (e.g. 95.42)


# Strip — one row per outstanding contract.
# Columns: symbol, root, expiry, settle, implied_rate, vs_ocr_bp
strip = pd.DataFrame(...)

# Reference rates — daily series, one column per source.
# Index: business-day DatetimeIndex
# Columns: effr, sofr
ref_rates = pd.DataFrame(...)

# FOMC calendar — list of meeting end-dates (datetime.date),
# ordered ascending. Filter to today onwards before use.
fomc_dates: list[date] = [...]
```

WHY THIS SHAPE

- Every later snippet in this playbook accepts these three objects as inputs. If you can produce them from your provider, you can run the whole thing.
- **Settle is in price space** (e.g. 95.42), not rate space. The implied-rate conversion happens once, on the next slide.
- **Expiry uses** `datetime.date` not `pd.Timestamp` — keeps day-of-month arithmetic simple in the meetings module.
- Open interest, volume, and bid/ask can be added as extra columns without breaking anything — every helper indexes by name.

IMPLIED RATE FROM PRICE

One line of arithmetic. The trick is interpreting which rate you've just produced.

```
implied_rate_pct = 100 - settlement_price
```

Both SR3 and ZQ are quoted in IMM/100-minus-rate convention.

WHAT THE RATE REPRESENTS

The price 100 - rate convention means a **contract trading at 95.42 implies a 4.58% rate** for the contract's reference period. The reference period differs by product:

- **SR3** — average *three-month* compounded SOFR over the contract's reference quarter.
- **ZQ** — simple arithmetic average of *daily* effective federal funds rates over the contract's calendar month.

The two are **not directly comparable bar-to-bar**. The Products tab keeps one shared OCR baseline (effective FFR) so the eye lines up the strip against the same reference regardless of which product is shown.

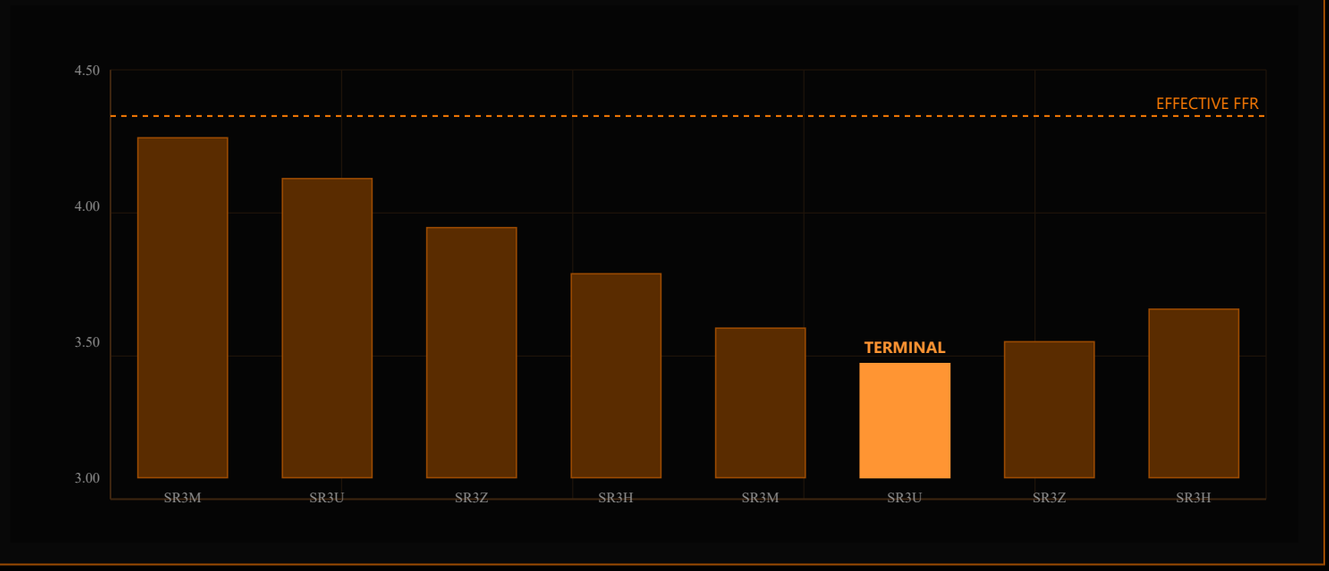
PYTHON

```python
def implied_rate(settle: float) -> float:
    """Convert a CME futures price to a rate in percent."""
    return 100.0 - settle


def add_implied(strip: pd.DataFrame, ocr: float) -> pd.DataFrame:
    """Attach implied_rate and vs_ocr_bp columns to a strip frame."""
    out = strip.copy()
    out["implied_rate"] = 100.0 - out["settle"]
    out["vs_ocr_bp"]    = (out["implied_rate"] - ocr) * 100.0
    return out
```

STRIP VIEW

One bar per outstanding contract. OCR drawn as a horizontal reference. Toggle SOFR / FF, same baseline.

STRIP VIEW (ILLUSTRATIVE)



PRODUCT TOGGLE

The Products tab shows a single toggle — `SOFR` / `FED FUNDS`. The **same OCR baseline** (effective FFR) is drawn on both views, which is what lets the user eyeball the spread between the two surfaces just by flipping the toggle.

On the SOFR view, also draw a small annotation showing the spot SOFR rate vs the spot EFFR — the basis usually lives in single digits of basis points but flares around quarter-ends.

PYTHON

```python
import plotly.graph_objects as go


def plot_strip(strip: pd.DataFrame, ocr: float, *,
               terminal_symbol: str | None = None) -> go.Figure:
    colors = [
        "#FF9533" if s == terminal_symbol else "#5A2C00"
        for s in strip["symbol"]
    ]
    fig = go.Figure(go.Bar(
        x=strip["symbol"], y=strip["implied_rate"],
        marker_color=colors, marker_line_color="#9A4A02",
        hovertemplate="%{x}<br>%{y:.3f}%<extra></extra>",
    ))
    fig.add_hline(y=ocr, line_dash="dash", line_color="#FE7C04",
                  annotation_text="EFFECTIVE FFR",
                  annotation_position="right",
                  annotation_font_color="#FE7C04")
    fig.update_layout(
        template="plotly_dark",
        paper_bgcolor="#000", plot_bgcolor="#050505",
        font=dict(family="Segoe UI", color="#D0D0D0"),
        yaxis_title="Implied rate (%)", xaxis_title=None,
        margin=dict(l=50, r=20, t=20, b=40),
    )
    return fig
```

DAY-WEIGHTED POST-MEETING RATE

Decompose each ZQ settle into pre- and post-meeting regimes. The math is one line; the bookkeeping is iterative.

```
monthly_avg = ((D-1) · prevRate + (N-D+1) · postRate) / N
```

Solve for postRate to recover the rate priced for the period AFTER the meeting.

```
postRate = (monthly_avg · N - (D-1) · prevRate) / (N - D + 1)
```

N = days in the month, D = meeting day-of-month, prevRate = rate priced before the meeting.

WHY DAY-WEIGHTING

A 30-day fed funds future settles to the **arithmetic average of the daily effective rate** across its expiry month. If the FOMC meets on day D of an N-day month, the contract's settle blends two regimes: the rate before the meeting, and the rate after.

Rearranging the average gives you the implied post-meeting rate. Iterate forward through the FOMC calendar — the post-meeting rate at meeting **k** becomes the prev-rate going into meeting **k+1**.

Edge case. When the next calendar month has its own ZQ contract and **no FOMC meeting**, that contract's rate **is** the post-meeting rate (constant for the whole month). This shortcut avoids noisy day-weighting for late-month meetings.

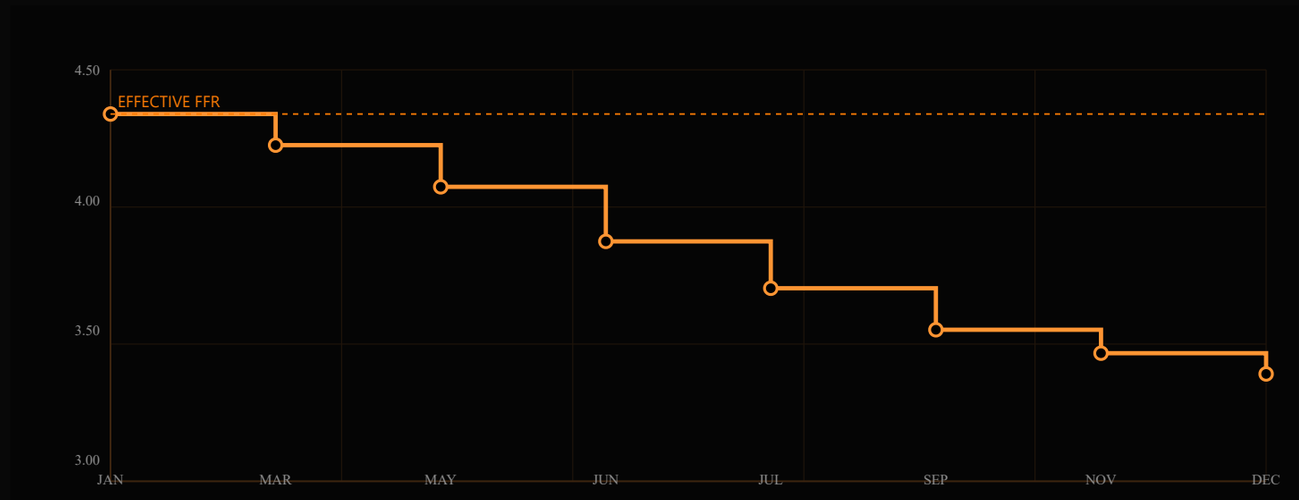
PYTHON

```python
from calendar import monthrange


def post_meeting_rate(contract_rate: float, prev_rate: float,
                      meeting_day: int, days_in_month: int) -> float:
    days_after = days_in_month - meeting_day + 1
    if days_after <= 0:
        return contract_rate
    return (contract_rate * days_in_month
            - (meeting_day - 1) * prev_rate) / days_after


def build_meeting_path(zq_by_month: dict[tuple[int, int], float],
                       effr_today: float,
                       fomc_dates: list[date]) -> list[dict]:
    """Iterate FOMC dates, chaining prev_rate forward."""
    prev = effr_today
    out = []
    for d in fomc_dates:
        rate = zq_by_month.get((d.year, d.month))
        if rate is None:
            continue
        N = monthrange(d.year, d.month)[1]
        # If the next month has a non-meeting ZQ, use its rate directly.
        next_key = (d.year + (d.month == 12), d.month % 12 + 1)
        next_rate = zq_by_month.get(next_key)
        next_has_meeting = any(
            fd.year == next_key[0] and fd.month == next_key[1]
            for fd in fomc_dates
        )
        if next_rate is not None and not next_has_meeting:
            post = next_rate
        else:
            post = post_meeting_rate(rate, prev, d.day, N)
        out.append({"meeting": d, "post_rate": post,
                    "cum_cuts": (effr_today - post) / 0.25})
        prev = post
    return out
```

PROBABILITY DISTRIBUTION

From an implied post-meeting rate, recover P(hold) / P(cut 25) / P(cut 50) / etc. — interpolation between 25 bp levels.

MEETING-PATH CHART (ILLUSTRATIVE)



READING THE CHART

The step chart plots the **post-meeting implied rate** at each FOMC date — output of the iterator on the previous slide. The **cumulative cuts** column quantifies how many 25 bp moves the curve has priced relative to today's EFFR.

The probability table rounds each post-meeting rate to the two nearest 25 bp grid points and assigns probability mass by the fractional distance — exactly what CME FedWatch publishes.

```
raw_cuts = (EFFR - postRate) / 0.0025
The integer part is the floor of the cuts implied; the fractional part splits probability mass.
```

PYTHON

```python
def meeting_probs(post_rate: float, effr: float) -> dict:
    """CME-FedWatch-style P(target rate) interpolation."""
    raw = (effr - post_rate) / 0.0025
    lower = int(raw // 1)
    frac  = raw - lower

    mass: dict[int, float] = {lower: 1 - frac}
    if frac > 0.001:
        mass[lower + 1] = frac

    return {
        "hold":   100 * mass.get(0,  0.0),
        "cut25":  100 * mass.get(1,  0.0),
        "cut50":  100 * mass.get(2,  0.0),
        "cut75":  100 * mass.get(3,  0.0),
        "hike25": 100 * mass.get(-1, 0.0),
    }
```

CALENDAR SPREAD MATRIX

Forward spreads from each contract to its +3M / +6M / +9M / +12M neighbour. Anchored on the terminal, with a front-contract fallback.

CALENDAR SPREAD MATRIX (ILLUSTRATIVE)

	+3M	+6M	+9M	+12M
FRONT (M+0)	-12bp	-25bp	-41bp	-58bp
M+3	-13bp	-29bp	-46bp	-54bp
M+6	-16bp	-33bp	-41bp	-38bp
TERMINAL	-17bp	-25bp	-22bp	-8bp
M+12	-8bp	-3bp	+9bp	+20bp
M+15	-2bp	+6bp	+15bp	+28bp

ANCHOR-SELECTION RULE

1. Find the **terminal** contract — the first peak (if hikes are priced) or first trough (if cuts are priced) on the strip relative to the OCR baseline.
2. If the strip is *monotonic* (no inflection found and the terminal sits at the last contract), *fall back* to the front contract as the anchor.
3. Compute spreads from the anchor to **+3M, +6M, +12M** forward. Repeat from each row's contract to build the matrix.
4. Render in basis points. Negative = lower forward rate (cuts priced); positive = higher (hikes priced).

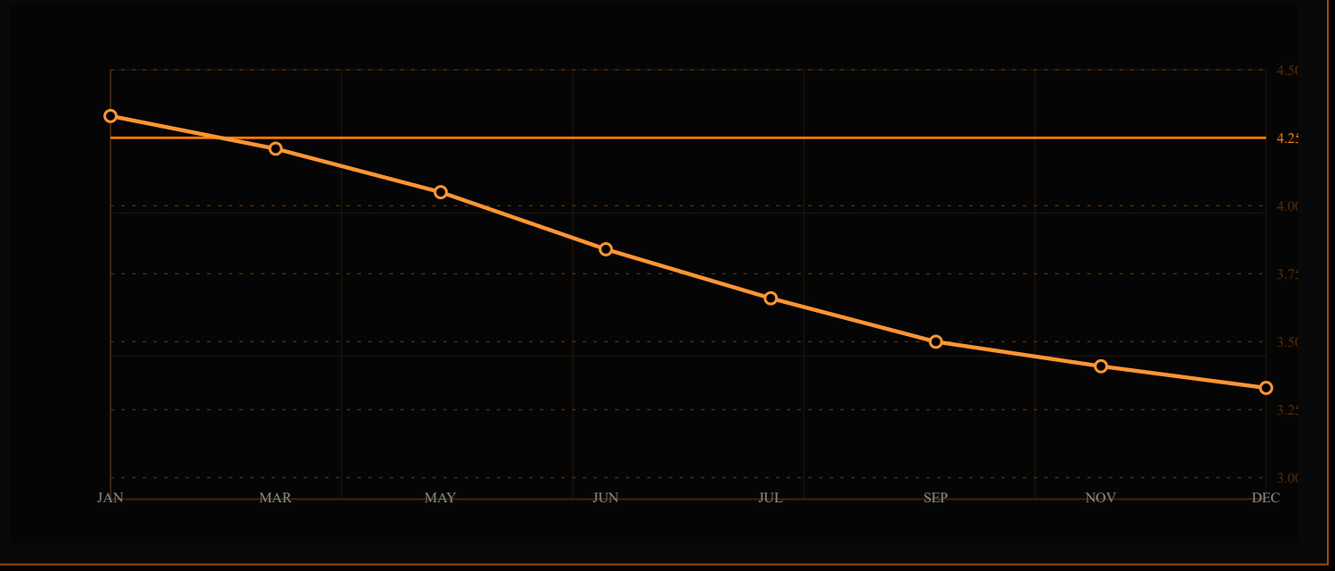
PYTHON

```python
def find_terminal(strip: pd.DataFrame, ocr: float) -> pd.Series:
    front = strip.iloc[0]
    hiking = front["implied_rate"] >= ocr
    best = front
    for _, row in strip.iloc[1:].iterrows():
        if hiking and row["implied_rate"] >= best["implied_rate"]:
            best = row
        elif not hiking and row["implied_rate"] <= best["implied_rate"]:
            best = row
        else:
            break
    return best


def spread_matrix(strip: pd.DataFrame, ocr: float,
                  horizons_m: tuple[int, ...] = (3, 6, 9, 12)) -> pd.DataFrame:
    term = find_terminal(strip, ocr)
    term_is_last = term["symbol"] == strip.iloc[-1]["symbol"]
    anchor = strip.iloc[0] if term_is_last else term
    anchor_mo = anchor["expiry"].month + 12 * anchor["expiry"].year

    rows: list[dict] = []
    for _, row in strip.iterrows():
        row_mo = row["expiry"].month + 12 * row["expiry"].year
        spreads = {}
        for h in horizons_m:
            target = row_mo + h
            fwd = strip[strip["expiry"].apply(
                lambda d: d.month + 12 * d.year >= target
            )]
            if fwd.empty:
                spreads[f"+{h}M"] = float("nan")
            else:
                spreads[f"+{h}M"] = round(
                    (fwd.iloc[0]["implied_rate"] - row["implied_rate"]) * 100
                )
        rows.append({"contract": row["symbol"], **spreads})
    return pd.DataFrame(rows).set_index("contract")
```

CB LVL INDICATOR

Reference rails at every 25 bp policy level. Settlement rail is highlighted; the rest fade into the background.

CB LVL VIEW (ILLUSTRATIVE)



WHAT IT DOES

CB LVL overlays the meeting-path chart with a grid of **policy-rate rails** at every 25 bp increment. The **settlement rail** (today's EFFR rounded to the nearest 25 bp) gets a brighter, solid line; surrounding rails are faint and dashed.

The result is a chart that reads cleanly even to readers who don't know fed funds futures: the implied path is plotted in price space, but the rails reveal which 25 bp level it's **nearest to** at any point in time.

PYTHON

```python
def cb_levels(effr: float,
              band_bp: int = 100,
              step_bp: int = 25) -> list[float]:
    settle = round(effr / 0.25) * 0.25
    n = band_bp // step_bp
    return [settle + (i - n) * (step_bp / 100.0)
            for i in range(2 * n + 1)]


def add_cb_levels(fig, levels: list[float], settle: float):
    for lv in levels:
        is_settle = abs(lv - settle) < 0.01
        fig.add_hline(
            y=lv,
            line_color="#FE7C04" if is_settle else "#5A2C00",
            line_dash="solid" if is_settle else "dot",
            line_width=1.4 if is_settle else 0.6,
            annotation_text=f"{lv:.2f}%"
                            + (" · SETTLE" if is_settle else ""),
            annotation_position="right",
            annotation_font=dict(
                color="#FE7C04" if is_settle else "#5A2C00",
                size=9, family="IBM Plex Mono",
            ),
        )
```

TERMINAL RATE

The pivotal contract on the strip. Detects regime (hiking vs cutting) from the front-vs-OCR sign, then walks for the first reversal.

TERMINAL DETECTION RULES

Define the strip's terminal as the first inflection point relative to the OCR baseline:

- If the front contract sits *above* OCR, hikes are priced — terminal is the **first local maximum** walking forward.
- If the front contract sits *below* OCR, cuts are priced — terminal is the **first local minimum**.
- Walk one contract at a time. Keep extending while the rate continues in the same direction; stop the moment it reverses.
- If the strip is monotonic (no reversal found), the algorithm returns the last contract. The downstream code on slide 10 handles this case by falling back to the front contract for spread anchoring.

Filter to active contracts only (settle > 0) before walking. Inactive contracts can poison the search with stale prints.

PYTHON

```python
def find_terminal(strip: pd.DataFrame, ocr: float) -> pd.Series:
    """Return the row of the first peak (hiking) or trough (cutting)."""
    active = strip[strip["settle"] > 0].reset_index(drop=True)
    if active.empty:
        return strip.iloc[0]

    front = active.iloc[0]
    hiking = front["implied_rate"] >= ocr

    best = front
    for _, row in active.iloc[1:].iterrows():
        if hiking and row["implied_rate"] >= best["implied_rate"]:
            best = row
        elif not hiking and row["implied_rate"] <= best["implied_rate"]:
            best = row
        else:
            break
    return best


# Spread label rule used in the spread matrix:
#   if terminal is the LAST contract on the strip → anchor on FRONT
#   else → anchor on TERMINAL
def anchor_for(strip: pd.DataFrame, ocr: float) -> pd.Series:
    term = find_terminal(strip, ocr)
    if term["symbol"] == strip.iloc[-1]["symbol"]:
        return strip.iloc[0]
    return term
```

PUTTING IT TOGETHER

One PDF — methodology in front, full runnable code in the appendix that follows. Drop into Claude Code with the prompt below.

BUILD CHECKLIST

1. Pull SR3 and ZQ daily settlement files from your provider for the next 18 months of contracts.
2. Pull EFFR and SOFR from the New York Fed CSV downloads — daily series, latest 12 months is plenty.
3. Hard-code the FOMC schedule for the next ~18 months from `federalreserve.gov`.
4. Conform the data to the schemas on slide 5.
5. Copy the appendix code into a single `.py` or `.ipynb` file. It runs as-is against the synthetic data.
6. Swap the three `make_mock_*` generators for your real loaders. Done.

PROMPT TEMPLATE FOR CLAUDE CODE

Open a fresh Claude Code session in any directory. Drop this PDF into the chat, then paste the prompt below — replacing `<your_provider>` with whatever data source you use (Polygon, Refinitiv, IBKR, your broker's API, CME DataMine, a CSV folder, anything).

PROMPT

```
I'm following the Capital Flows Research STIR
Replication Playbook (PDF attached). The
runnable code lives in the appendix at the
back of the same PDF.

My data source is <your_provider>.

Please:
  1. Re-create the appendix code as a single
     Python file in this directory.
  2. Replace the make_mock_* loaders with real
     loaders that hit my provider, producing
     the schemas defined on slide 5.
  3. Run the end-to-end driver and show me
     the four Plotly charts.
```

CAPITAL FLOWS RESEARCH

CODE APPENDIX

SIX LISTINGS · RUNS END-TO-END ON SYNTHETIC DATA · DROP-IN LOADERS

Concatenate the next six listings in order to get a single self-contained Python program. Replace the three `make_mock_*` functions in A2 with real loaders that hit your data source — every other line stays the same.

A1 · IMPORTS, PALETTE, SCHEMAS

Foundations every later listing depends on. Drop straight in — no provider connection yet.

PYTHON

```python
# ── A1 · IMPORTS, PALETTE, SCHEMAS ──────────────────────────────────────
from __future__ import annotations
from calendar import monthrange
from dataclasses import dataclass
from datetime import date, timedelta
import numpy as np, pandas as pd
import plotly.graph_objects as go

# Brand palette — used by every Plotly figure further down.
CFR = {
    "bg":     "#000000", "panel":     "#080808", "rule":      "#3D2510",
    "orange": "#FE7C04", "orangeHot": "#FF9533", "orangeDim": "#5A2C00",
    "text":   "#D0D0D0", "green":     "#00E676", "red":       "#FF1744",
}

# CME month-code map → builds public-standard symbols (e.g. SR3M5, ZQK6).
_CME_MONTH_CODES = {1:"F",2:"G",3:"H",4:"J",5:"K",6:"M",
                    7:"N",8:"Q",9:"U",10:"V",11:"X",12:"Z"}

def _cme_symbol(root: str, expiry: date) -> str:
    return f"{root}{_CME_MONTH_CODES[expiry.month]}{expiry.year % 10}"


# Single contract (one row in the strip DataFrame).
@dataclass
class Contract:
    symbol: str   # e.g. "SR3M5", "ZQK6"
    root:   str   # "SR3" (3-month SOFR) or "ZQ" (30-day Fed Funds)
    expiry: date  # last day of the contract month
    settle: float # last settlement price (e.g. 95.42)


def to_strip(contracts: list[Contract]) -> pd.DataFrame:
    """Wrap a list of Contract into the canonical strip DataFrame."""
    return pd.DataFrame([c.__dict__ for c in contracts])
```

A2 · SYNTHETIC DATA · (REPLACE WITH PROVIDER LOADERS)

Three loaders. Output schemas match slide 5. Swap each with a function that hits your data source.

PYTHON

```python
# ── A2 · SYNTHETIC DATA — REPLACE with your provider's loaders ──────────────
# Three loaders.  Output schemas are documented on slide 5 of the playbook.
# Drop in your data feed (Refinitiv, Polygon, IBKR, CME DataMine, CSVs, …)
# and produce DataFrames with the same column names — every other line
# in this appendix stays identical.

def make_mock_strip(today: date) -> pd.DataFrame:
    contracts: list[Contract] = []
    # SOFR 3-month future — quarterly listings out two years
    sr3_months = [(today.year + (today.month + i*3 - 1) // 12,
                   ((today.month - 1 + i*3) % 12) + 1) for i in range(8)]
    sr3_rates = [4.20, 4.05, 3.92, 3.75, 3.55, 3.42, 3.50, 3.62]
    for (y, m), r in zip(sr3_months, sr3_rates):
        qm = next(q for q in (3,6,9,12) if q >= m) if m not in (3,6,9,12) else m
        exp = date(y, qm, monthrange(y, qm)[1])
        contracts.append(Contract(_cme_symbol("SR3", exp), "SR3", exp, 100.0 - r))
    # Fed Funds 30-day future — monthly listings out 18 months
    for i, r in enumerate(np.linspace(4.30, 3.30, 18) + np.random.normal(0, 0.015, 18)):
        m = ((today.month - 1 + i) % 12) + 1
        y = today.year + (today.month + i - 1) // 12
        exp = date(y, m, monthrange(y, m)[1])
        contracts.append(Contract(_cme_symbol("ZQ", exp), "ZQ", exp, 100.0 - r))
    return to_strip(contracts).sort_values(["root", "expiry"]).reset_index(drop=True)


def make_mock_ref_rates(today: date, days: int = 90) -> pd.DataFrame:
    """EFFR + SOFR daily reference rates (NY Fed publishes these as CSVs)."""
    idx  = pd.bdate_range(end=today, periods=days)
    effr = pd.Series(4.33, index=idx) + np.random.normal(0, 0.005, days)
    sofr = effr + np.random.normal(0.005, 0.005, days)   # tiny basis
    return pd.DataFrame({"effr": effr, "sofr": sofr})


def make_mock_fomc_dates(today: date) -> list[date]:
    """Hand-maintained: refresh from federalreserve.gov each year."""
    cursor, out = today + timedelta(days=15), []
    for _ in range(8):
        out.append(cursor); cursor = cursor + timedelta(days=46)
    return out
```

A3 · IMPLIED RATE · TERMINAL · STRIP

Price → rate, regime detection, the Plotly view used by the Products tab.

PYTHON

```python
# ── A3 · IMPLIED RATE · TERMINAL · STRIP VIEW ──────────────────────────
def implied_rate(settle: float) -> float:
    return 100.0 - settle


def add_implied(strip: pd.DataFrame, ocr: float) -> pd.DataFrame:
    out = strip.copy()
    out["implied_rate"] = 100.0 - out["settle"]
    out["vs_ocr_bp"]    = (out["implied_rate"] - ocr) * 100.0
    return out


def find_terminal(strip_view: pd.DataFrame, ocr: float) -> pd.Series:
    """First peak (hiking) or trough (cutting) on the strip relative to OCR."""
    active = strip_view[strip_view["settle"] > 0].reset_index(drop=True)
    if active.empty:
        return strip_view.iloc[0]
    front  = active.iloc[0]
    hiking = front["implied_rate"] >= ocr
    best   = front
    for _, row in active.iloc[1:].iterrows():
        if hiking and row["implied_rate"] >= best["implied_rate"]:
            best = row
        elif not hiking and row["implied_rate"] <= best["implied_rate"]:
            best = row
        else:
            break
    return best


def plot_strip(strip_view: pd.DataFrame, ocr: float, title: str) -> go.Figure:
    term = find_terminal(strip_view, ocr)
    colors = [CFR["orangeHot"] if s == term["symbol"] else CFR["orangeDim"]
              for s in strip_view["symbol"]]
    fig = go.Figure(go.Bar(
        x=strip_view["symbol"], y=strip_view["implied_rate"],
        marker_color=colors, marker_line_color="#9A4A02",
        hovertemplate="%{x}<br>%{y:.3f}%<extra></extra>",
    ))
    fig.add_hline(
        y=ocr, line_dash="dash", line_color=CFR["orange"],
        annotation_text="EFFECTIVE FFR", annotation_position="right",
        annotation_font=dict(color=CFR["orange"], family="Segoe UI"),
    )
    fig.update_layout(
        title=dict(text=title, font=dict(color=CFR["orange"], family="Bahnschrift", size=20)),
        template="plotly_dark", paper_bgcolor=CFR["bg"], plot_bgcolor="#050505",
        font=dict(family="Segoe UI", color=CFR["text"]),
        yaxis_title="Implied rate (%)", xaxis_title=None,
        margin=dict(l=60, r=20, t=60, b=40), height=420,
    )
    return fig
```

A4 · MEETING-PATH MATH

Day-weighted post-meeting rate, FOMC iterator, FedWatch-style probability mass.

PYTHON

```python
# ── A4 · MEETING-PATH MATH · PROBABILITIES ──────────────────────────────
def post_meeting_rate(contract_rate: float, prev_rate: float,
                      meeting_day: int, days_in_month: int) -> float:
    """Recover the rate priced for the period AFTER an FOMC meeting from the
    monthly-average ZQ contract.  monthly_avg = ((D-1)*prev + (N-D+1)*post) / N"""
    days_after = days_in_month - meeting_day + 1
    if days_after <= 0:
        return contract_rate
    return (contract_rate * days_in_month - (meeting_day - 1) * prev_rate) / days_after


def build_meeting_path(zq_strip: pd.DataFrame, effr_today: float,
                       fomc_dates: list[date]) -> pd.DataFrame:
    zq_by_month = {(r["expiry"].year, r["expiry"].month): r["implied_rate"]
                   for _, r in zq_strip.iterrows()}
    fomc_keys = {(d.year, d.month) for d in fomc_dates}
    prev = effr_today
    rows: list[dict] = []
    for d in fomc_dates:
        rate = zq_by_month.get((d.year, d.month))
        if rate is None:
            continue
        N = monthrange(d.year, d.month)[1]
        ny, nm = (d.year + (d.month == 12), d.month % 12 + 1)
        next_rate = zq_by_month.get((ny, nm))
        next_has_meeting = (ny, nm) in fomc_keys
        if next_rate is not None and not next_has_meeting:
            post = next_rate                                  # next-month shortcut
        else:
            post = post_meeting_rate(rate, prev, d.day, N)
        rows.append({"meeting": d, "post_rate": post,
                     "cum_cuts": (effr_today - post) / 0.25})
        prev = post
    return pd.DataFrame(rows)


def meeting_probs(post_rate: float, effr: float) -> dict[str, float]:
    """CME-FedWatch-style P(target rate) — interpolation between 25 bp levels."""
    raw = (effr - post_rate) / 0.25
    lower = int(np.floor(raw))
    frac  = raw - lower
    mass: dict[int, float] = {lower: 1 - frac}
    if frac > 0.001:
        mass[lower + 1] = frac
    return {"hold":   100 * mass.get(0,  0.0),
            "cut25":  100 * mass.get(1,  0.0),
            "cut50":  100 * mass.get(2,  0.0),
            "cut75":  100 * mass.get(3,  0.0),
            "hike25": 100 * mass.get(-1, 0.0)}
```

Spread matrix, the meeting-path Plotly view, the CB LVL overlay built on top of it.

PYTHON

```python
# ── A5 · SPREAD MATRIX · MEETING-PATH PLOT · CB LVL OVERLAY ──────────────
def spread_matrix(strip_view: pd.DataFrame, ocr: float,
                  horizons_m: tuple[int, ...] = (3, 6, 9, 12)) -> pd.DataFrame:
    if strip_view.empty:
        return pd.DataFrame()
    rows: list[dict] = []
    for _, row in strip_view.iterrows():
        row_mo = row["expiry"].month + 12 * row["expiry"].year
        spreads: dict[str, float] = {}
        for h in horizons_m:
            target = row_mo + h
            forward = strip_view[strip_view["expiry"].apply(
                lambda d: d.month + 12 * d.year >= target)]
            spreads[f"+{h}M"] = (round((forward.iloc[0]["implied_rate"]
                                        - row["implied_rate"]) * 100)
                                 if not forward.empty else float("nan"))
        rows.append({"contract": row["symbol"], **spreads})
    return pd.DataFrame(rows).set_index("contract")


def plot_meeting_path(path: pd.DataFrame, effr: float) -> go.Figure:
    fig = go.Figure(go.Scatter(
        x=path["meeting"], y=path["post_rate"], mode="lines+markers",
        line=dict(color=CFR["orangeHot"], width=2.4, shape="hv"),
        marker=dict(color=CFR["bg"],
                    line=dict(color=CFR["orangeHot"], width=1.5), size=8),
    ))
    fig.add_hline(y=effr, line_dash="dash", line_color=CFR["orange"],
                  annotation_text="EFFECTIVE FFR", annotation_position="right",
                  annotation_font=dict(color=CFR["orange"]))
    fig.update_layout(template="plotly_dark", paper_bgcolor=CFR["bg"],
                      plot_bgcolor="#050505",
                      font=dict(family="Segoe UI", color=CFR["text"]),
                      margin=dict(l=60, r=40, t=60, b=40), height=420)
    return fig


def cb_levels(effr: float, band_bp: int = 100, step_bp: int = 25) -> list[float]:
    settle = round(effr / 0.25) * 0.25
    n = band_bp // step_bp
    return [settle + (i - n) * (step_bp / 100.0) for i in range(2 * n + 1)]


def plot_cb_lvl(path: pd.DataFrame, effr: float) -> go.Figure:
    fig    = plot_meeting_path(path, effr)
    settle = round(effr / 0.25) * 0.25
    for lv in cb_levels(effr, band_bp=150):
        is_settle = abs(lv - settle) < 0.01
        fig.add_hline(
            y=lv,
            line_color=CFR["orange"] if is_settle else CFR["orangeDim"],
            line_dash="solid"  if is_settle else "dot",
            line_width=1.4 if is_settle else 0.6,
        )
    return fig
```

A6 · END-TO-END DRIVER

Run last. Replace the three make_mock_* calls in step 1 with real loaders — every other line stays the same.

PYTHON

```python
# ── A6 · END-TO-END DRIVER ──────────────────────────────────────────────
# Concatenate appendices A1-A5 above into a single .py file, then run this driver.
# To plug in your data provider, replace the three make_mock_* calls in step 1.

if __name__ == "__main__":
    np.random.seed(42)
    TODAY = date.today()

    # 1. Load inputs  ── REPLACE these three lines with your provider's loaders ──
    strip      = make_mock_strip(TODAY)
    ref_rates  = make_mock_ref_rates(TODAY)
    fomc_dates = make_mock_fomc_dates(TODAY)

    # 2. Anchor rates
    OCR  = float(ref_rates["effr"].iloc[-1])
    SOFR = float(ref_rates["sofr"].iloc[-1])
    print(f"OCR (EFFR) today: {OCR:.4f}%   SOFR spot: {SOFR:.4f}%   "
          f"basis {(SOFR-OCR)*100:+.1f} bp")

    # 3. Decorate strip + split by product
    strip      = add_implied(strip, OCR)
    sofr_strip = strip[strip["root"] == "SR3"].reset_index(drop=True)
    ff_strip   = strip[strip["root"] == "ZQ"].reset_index(drop=True)

    # 4. PRODUCTS tab — strip views (toggleable in your UI)
    plot_strip(sofr_strip, OCR, "PRODUCTS · SOFR (SR3) STRIP").show()
    plot_strip(ff_strip,   OCR, "PRODUCTS · FED FUNDS (ZQ) STRIP").show()

    # 5. MEETINGS · STRIP — implied post-meeting rate path
    path = build_meeting_path(ff_strip, OCR, fomc_dates)
    plot_meeting_path(path, OCR).show()

    # 6. MEETINGS · STRIP — probability table
    probs_df = pd.DataFrame(
        [meeting_probs(r, OCR) for r in path["post_rate"]],
        index=[d.isoformat() for d in path["meeting"]],
    ).round(1)
    print(probs_df)

    # 7. MEETINGS · SPREADS — calendar matrix
    print(spread_matrix(ff_strip, OCR))

    # 8. MEETINGS · CB LVL — meeting path overlaid with policy rails
    plot_cb_lvl(path, OCR).show()
```